

Chapter 14

Closed Captioning, Subtitling, and Teletext

14.1. INTRODUCTION

One of the major advantages offered by digital television is the ability to augment programs with a variety of text, graphics, and even subsidiary audio–visual information. The simplest of these ancillary services are closed captioning, also known as subtitling and teletext.

Subtitling and teletext services in digital television extend those provided in analog television, which make use of the otherwise unused transmission capacity in the vertical blanking interval (VBI) to transmit text and simple graphics. Very low coding efficiencies are obtained when this information in the VBI is coded using the video coding tools of MPEG-2. Therefore, both DVB and ATSC carry this ancillary text information in special-purpose data channels.

14.2. DVB SUBTITLES AND TELETEXT

The ancillary services provided by DVB are as follows:

- *subtitling*, which provides a similar closed-captioning service to that provided by analog television but with significantly greater flexibility and
- *teletext*, which replicates the analog television teletext service based on System-B teletext and is primarily intended to provide backward compatibility with previous systems and will eventually be superseded by datacasting.

This section describes both of these services.

Both subtitling and teletext data in a DVB stream are carried in PES packets. Table 14.1 shows the information used by a decoder to identify subtitling and teletext data so that it can be passed to the appropriate decoder. The **stream_type** field of the program map table entry pointing to these PES packets identifies the stream as PES packets containing private data. The headers of these PES packets also identify the payload as private data (by setting the **stream_id** field to the value 0xBD). DVB

Table 14.1 Identification of subtitling and teletext data in DVB.

Field	Value for subtitle	Value for teletext
stream_type in PMT	0x06	0x06
stream_id in PES header	0xBD	0xBD
data_identifier in PES payload	0x20	0x10 – 0x1F

has a convention that the first payload byte of a PES packet containing private data carries the **data_identifier** field, which is used to identify uniquely the type of data in the payload of the packet.

Further information about subtitling and teletext streams is provided to the decoder in the component, subtitling, and teletext descriptors described in Chapter 12.

14.2.1. Subtitles

The subtitling function of DVB is designed for carrying the simple text and graphics required for closed captioning. These services are used traditionally to provide text for the hearing impaired and in lieu of dubbing for foreign language content.

Subtitling in DVB is defined in ETS 300 743 [1], which specifies the transmission and coding of graphical elements. Data is transmitted in the form of pixel structures, which define objects that can be displayed. These pixel structures are reusable but are not suitable for downloading character sets because there is no special error protection provided for pixel structures, which leads to error expansion when an incorrectly received pixel structure is reused. DVB does not provide any specific character set or font for subtitles. A means is provided to utilize ROM-based pixel structures, however, although no ROM-based pixel structure is defined by the standard. A network operator may use ROM-based pixel structures for character generation.

In DVB subtitling, a *region* is composed of one or more objects present in the decoder’s memory. A group of regions that are displayed simultaneously forms a *page*.

Color information is coded with a resolution of 2, 4, or 8 bits, requiring color lookup tables with 4, 16, or 256 colors, respectively. Pixel data is compressed by applying run-length coding to consecutive pixels of the same color.

14.2.1.1. Timing

DVB subtitles are delivered in the MPEG-2 stream as private data, carried in PES packets. The MPEG-2 presentation time stamp defines the display time for a subtitle. This allows subtitle display to be synchronized with other elements of the displayed program including video and audio.

14.2.1.2. Example Subtitle Operation

A simple example of the operation of the DVB subtitling system is shown in the figures below. Figure 14.1 shows the initial (empty) display. The subtitling systems’

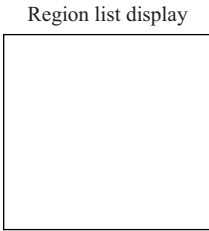


Figure 14.1 Initial display.

internal region list is represented on the left side and the output on the display on the right side.

Figure 14.2 shows the contents of the region list and display after the introduction of regions. Placing region A in the page specifies that this region is to be displayed. Regions outside the page (B and C) are not displayed. In this example, region A will be used to display a logo, whereas buffers B and C will be used to display text. Although the appearance of this logo is similar to the watermarks used in analog television, DVB subtitling does not provide a satisfactory means for conveying watermarks. A key characteristic of a watermark is that it is difficult to remove from the watermarked image. Because it is transmitted separately from the video, a DVB subtitle is easily removed.

Once the regions have been defined, the pixel structures defining graphics to be displayed can be downloaded as illustrated in Figure 14.3. Because region A is contained within the current version of the displayed page, the logo is displayed immediately after it is downloaded.

Figure 14.4 shows the delivery of the first text. This text is placed in the region B, but is not displayed immediately, because region B is not defined to lie within the page.

When the time for display of the first text arrives (as indicated by its associated presentation time stamp), a new version of the page description is activated in which region B is defined to lie at a particular location within the display. Although this text is displayed (Fig. 14.5), the next text to be displayed can be downloaded into region C.

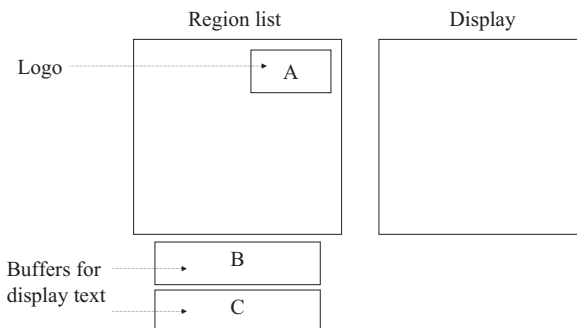


Figure 14.2 Introduce regions.

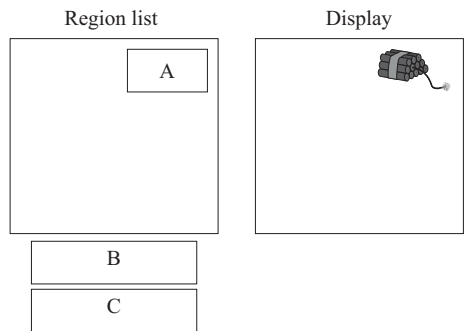


Figure 14.3 Deliver and display logo.

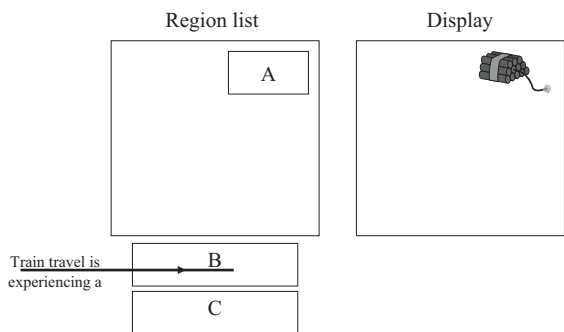


Figure 14.4 Deliver first text.

The second text is displayed by the use of a new page definition, in which region B is outside the display, but region C is now inside the display. Once the second text is displayed (Fig. 14.6), the third text can be downloaded into region B.

Finally as shown in Figure 14.7, the third text can be displayed.

This example illustrates the use of regions for immediate display (A) and the use of regions (B and C) for deferred display and simultaneous offscreen download of updates.

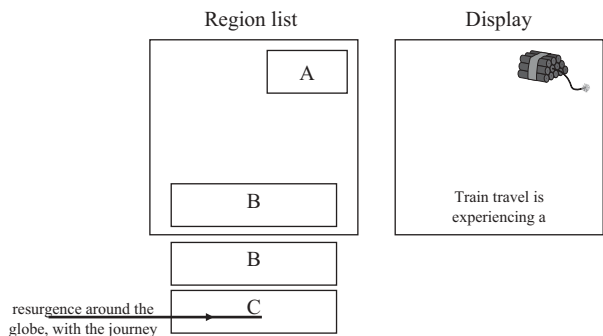


Figure 14.5 Reveal first text and deliver second text.

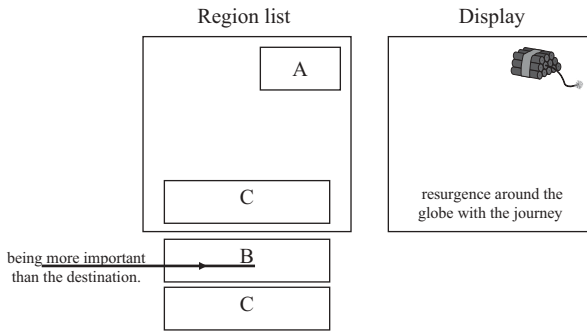


Figure 14.6 Reveal second text and deliver third text.

14.2.1.3. Coding for DVB Subtitling Data

Four new syntactic structures are defined to carry the DVB subtitling data. The *page composition segment* (PCS) defines the locations within the picture of those regions that are to be displayed as part of the page. Each region is defined by a *region composition segment*, which specifies the objects that make up that region. The structure of each object is defined by an *object data segment*, which specifies either the pixel structure for the object or one or more characters from a character set that make up the object. The *color lookup table* (CLUT) defines the 2, 4, or 8-bit colormap to be used for a particular page.

Page Composition Segment The PCS defines regions within the displayed picture into which data is subsequently loaded. Each region is specified to lie within a specific page. The PCS defines the following for one page:

- its **page_id**, which uniquely identifies the page;
- the **page_time_out**, which specifies a time in seconds after which the page is to be erased, protecting against the loss of a subsequent PCS that would cause this erasure;

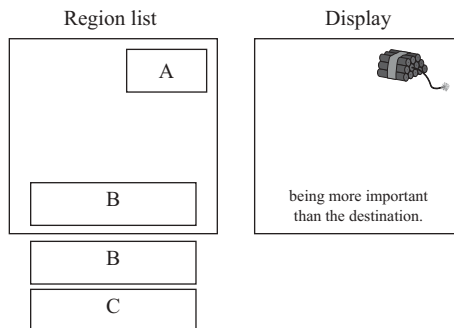


Figure 14.7 Reveal third text.

- the **page_version_number**, which is increased by 1 (mod 16) each time the page is modified;
- the **page_state**, which is used to specify where sufficient information is provided to permit random access and those points at which the memory plan of the decoder is to be changed;
- a list of regions (specified by their **region_id**) and their horizontal and vertical addresses in the display.

Region Composition Segment The region composition table defines for a particular region

- the **region_id**, which is used to uniquely identify the region;
- the **region_version_number**, which is increased by 1 (mod 16) each time the contents of the region are changed;
- the **region_fill_flag** and **region_fill_n-bit_color**, which are used to allow a region to be filled with a particular color before composition of objects begins;
- the **region_width** and **region_height**, specifying the horizontal and vertical size of the region, respectively;
- the **region_depth**, specifying the maximum pixel depth to be used in rendering the region and the **region_level_of_compatibility**, specifying the minimum CLUT size to be used for rendering the region;
- a list of objects from which the region is composed, each of which is identified by its unique **object_id** and their locations within the region.

Object Data Segment The object data segment is used to download data into a specified region in a particular page. Where a predefined character set is used, object data takes the form of indices in this character set and is carried directly in the object data segment. Where pixel-based subtitling is to occur, pixel data is carried within a number of pixel-data subblocks. The syntax of the object data segment is shown in Table 14.2.

The **sync_byte** has value “0000 1111,” and is used to ensure that correct synchronization is maintained during decoding. The value of **segment_type** for the object data segment is 0x13. The **page_id** field identifies the page in which this object appears, whereas the **segment_length** specifies the length in bytes of the remainder of the object data segment.

The **object_id** uniquely identifies this object, with the **object_version_number** used to distinguish between different versions of the same object. The **object_version_number** is increased by 1 each time the content of an object is modified.

The **object_coding_method** specifies whether the object is coded as a bitmap (“00”) or string of characters (“01”). The **non_modifying_color_flag** is set to 1 if the CLUT entry 1 does not overwrite other objects.

Table 14.2 Syntax for object data segment.

Syntax	Number of bits	Mnemonic
<code>object_data_segment() {</code>		
sync_byte	8	bslbf
segment_type	8	bslbf
page_id	16	bslbf
segment_length	16	uimsbf
object_id	16	bslbf
object_version_number	4	uimsbf
object_coding_method	2	bslbf
non_modifying_color_flag	1	bslbf
reserved	1	bslbf
if (object_coding_method=='00') {		
top_field_block_length	16	uimsbf
bottom_field_block_length	16	uimsbf
while(processed_length<top_field_block_length)		
pixel-data_sub-block()		
while(processed_length<bottom_field_block_length)		
pixel-data_sub-block()		
if (!wordaligned())		
"0000 0000"	8	bslbf
}		
if (object_coding_method=='01'){		
number_of_codes	8	uimsbf
for (i=1, i<=number_of_codes, i++)		
character_code	16	bslbf
}		
}		

© European Telecommunications Standards Institute 1997. © European Broadcasting Union 1997.
Further use, modification, redistribution is strictly prohibited. ETSI standards are available from
<http://pda.etsi.org/pda/> and http://www.etsi.org/services_products/freestandard/home.htm.

Where the object is coded as a bitmap, that is, pixel data is to be transmitted explicitly, the **object_coding_method** is transmitted as "00." Pixel data for the two fields of the interlaced display is transmitted separately. The number of bytes of data present for the top field is specified by **top_field_block_length**, whereas the number of bytes of data for the bottom field is specified by the value of **bottom_field_block_length**.

Specifying the value "01" for the **object_coding_method** field tells the decoder that the object makes use of a predefined character set. In this case, the number of characters in the object is determined by the value of **number_of_codes** and the values of the characters by the contents of the **character_code** fields. Because character

sets are not standardized, but are agreed privately between network operators and equipment manufacturers, these characters are not restricted to any particular code.

Each call to pixel-data_subblock() returns pixel values for one line of pixel data in the object.

EXAMPLE 14.1—Object Data Segment

Table 14.3 shows the makeup of an object data segment for an object with **page_id** equal to 0x10, **object_id** equal to 0x23, **object_version_number** 2, and coded as characters. The contents of the object are the character string “HELLO WORLD,” represented as ASCII characters.

Table 14.3 Example object data segment.

Field name	Field value (binary)	Meaning
sync_byte	0000 1111	Sync byte is always 0000 1111.
segment_type	0001 0011	0x13 means that this segment is an object data segment.
page_id	0000 0000 0001 0000	Page id is 0x10.
segment_length	0000 0000 0001 1011	27 data bytes follow in this object data segment.
object_id	0000 0000 0010 0011	Object id is 0 × 23.
object_version_number	0010	Object version number is 2.
object_coding_method	01	Object coded as characters.
non_modifying_color_flag	0	CLUT entry 1 modifies underlying objects.
Reserved	0	Always 0.
number_of_codes	0000 0000 0000 1011	Eleven characters follow.
character_code	0000 0000 0100 1000	“H”
character_code	0000 0000 0100 0101	“E”
character_code	0000 0000 0100 1100	“L”
character_code	0000 0000 0100 1100	“L”
character_code	0000 0000 0100 1111	“O”
character_code	0000 0000 0010 0000	“ ”
character_code	0000 0000 0101 0111	“W”
character_code	0000 0000 0100 1111	“O”
character_code	0000 0000 0101 0010	“R”
character_code	0000 0000 0100 1100	“L”
character_code	0000 0000 0100 0100	“D”

The bit stream representing this segment is therefore 0000 1111 0001 0011 0000 0000 0001 0000 0000 0000 0001 1011 0000 0000 0010 0011 0010 0100 0000 0000 0000 0000 1011 0000 0000 0100 1000 0000 0000 0100 0101 0000 0000 0100 1100 0000 0000 0100 1100 0000 0000 0100 1111 0000 0000 0101 0111 0000 0000 0100 1111 0000 0000 0101 0000 0000 0100 1100 0000 0000 0100 0100.



14.2.1.4. Pixel-Data Subblock

The pixel-data subblock carries pixel-based subtitling data, that is, all subtitling data not based on a predefined character set. The subset of the syntax of the pixel-data_sub-block required to transmit pixel data for an 8-bit color lookup table is shown in Table 14.4.

The **sync_byte**, **segment_type**, **page_id**, and **segment_length** fields have the same meaning as for the object data segment. The **data_type** field specifies the pixel depth of the color lookup table using the value 0x12 for an 8-bit table.

The **8-bit/pixel_code_string** field uses a run-length code to specify a run of pixels of a particular value in an 8-bit color lookup table. The possible values of the variable length code used for the **8-bit/pixel_code_string** field and their meanings are shown in Table 14.5. The length of each of these possible values is 16 or 24 bits, both of which are a multiple of 8, ensuring that whole bytes of data are always generated. This avoids the need for padding of incomplete bytes at the end of a line of pixels and simplifies the implementation of the decoder.

Using the values shown in Table 14.5, 1 pixel whose index in the color lookup table is 12 would be represented by the 8-bit code 0000 1100. A string of 17 consecutive pixels with color index 34 would be represented by the 24-bit code 0000 0000 1001 0001 0010 0010.

Table 14.4 Syntax for pixel-data_sub-block.

Syntax	Number of bits	Mnemonic
pixel-data_sub-block() {		
sync_byte	8	bslbf
segment_type	8	bslbf
page_id	16	bslbf
segment_length	16	uimsbf
data_type	8	bslbf
...		
if (data_type=='0x12') {		
repeat {		
8-bit/pixel_code_string		vlclbf
} until (next_bits == '0000 0000 0000 0000')		
0000 0000 0000 0000	16	bslbf
}		
...		
}		

Table 14.5 Eight bits per **pixel_code_string**.

Value	Meaning
0000 0001 – 1111 1111	One pixel in color 1 to one pixel in color 255.
0000 0000 0LLL LLLL	L (1–127) pixels in color 0.
0000 0000 1LLL LLLL CCCC CCCC	L (3–127) pixels in color C.

EXAMPLE 14.2—Pixel-Data Subblock

Table 14.6 shows the fields making up a pixel-data subblock to carry a line of 15 pixels whose indices in the current color lookup table are 0 0 0 0 0 5 10 10 10 10 10 10 0 5 10.

Table 14.6 Example pixel-data subblock.

Field name	Field value (binary)	Meaning
sync_byte	0000 1111	Sync byte is always 0000 1111.
segment_type	0001 0011	0x13 means that this segment is an object data segment.
page_id	0000 0000 0001 0000	Page id is 0x10.
segment_length	0000 0000 0000 1101	Thirteen data bytes follow.
data_type	0001 0010	0x12 means 8-bit/pixel color lookup table.
8-bit/pixel_code_string	0000 0000 0000 0101	Five pixels of color 0.
8-bit/pixel_code_string	0000 0101	One pixel of color 5.
8-bit/pixel_code_string	0000 0000 1000 0110 0000 1010	Six pixels of color 10.
8-bit/pixel_code_string	0000 0000 0000 0001	One pixel of color 0.
8-bit/pixel_code_string	0000 0101	One pixel of color 5.
8-bit/pixel_code_string	0000 1010	One pixel of color 10.
	0000 0000 0000 0000	End of pixel data.

The bit stream produced to carry this pixel-data subblock is therefore 0000 1111 0001 0011 0000 0000 0001 0000 0000 0000 0000 1101 0001 0010 0000 0000 0000 0101 0000 0101 0000 0000 1000 0110 0000 1010 0000 0000 0000 0001 0000 0101 0000 1010 0000 0000 0000 0000. ■

14.2.1.5. Color Lookup Table

The color lookup table specifies the color to be associated with each index in the color table. For each index, the luminance, two chrominance components, and transparency are defined.

14.2.2. Teletext

The DVB teletext service is intended to carry teletext coded in accordance with the ITU-R System B Teletext recommendation [2] and is defined by EN 300 472 [3]. The major purpose of this part of the standard is to provide backward compatibility with existing equipment, belonging to both the network operator and their customers. In this section, the characteristics of ITU-R System B Teletext are described, followed by a discussion of the carriage of this data in a DVB stream.

14.2.2.1. ITU-R System B Teletext

Teletext provides a text-based service that allows users to browse pages of information. Each page consists notionally of 25 lines, each of which contains 40 characters. Up to 99 pages may exist in one magazine. Up to eight magazines are supported. Teletext is based on a data carousel, which means that data is transmitted continuously, with new copies of all data transmitted regularly. A new copy of a page usually completely replaces its previous contents. By regularly retransmitting all pages, a receiver need to only have a very limited storage capability, certainly not sufficient to hold the whole data carousel.

All teletext data is transmitted in packets of 360 bits organized as 45 bytes, transmitted LSB first. In analog television, all teletext data is transmitted in the VBI. Up to 16 lines per field (32 lines per picture) are allocated for this purpose, with each teletext packet occupying one VBI line. The maximum average raw data rate is therefore 11.52 kbit per picture or 288 kbit/s. There are three types of packet: the page header packet, normal packets intended for direct display, and nondisplayable packets.

ITU-R System B Teletext Packet Header Packets begin with a 2-byte clock run-in sequence (0x5555) and a 1-byte framing code (0x27). Each packet header specifies a 3-bit magazine number and 5-bit line number to which the content of the packet is applicable. Page header packets also identify a page number within the magazine, a page subcode, and control information. All subsequent packets with the same magazine number refer to this page until another page header packet with the same magazine number is received.

For packet numbers between 0 and 24, the packet number indicates the line of the display on which the packet's data should be displayed. Packet numbers larger than 24 are used to transmit nondisplayed control data and are described fully in ETS 300 706.

Teletext data is always transmitted least-significant-bit-first.

The magazine and packet number fields are protected against errors using a Hamming (8,4) code. The Hamming (8,4) code is generated as a Hamming 7/4 code with an additional parity bit to ensure that the byte has odd parity. The 4 parity bits

produced are as follows:

$$\begin{aligned} P_1 &= 1 \oplus D_1 \oplus D_3 \oplus D_4 \\ P_2 &= 1 \oplus D_1 \oplus D_2 \oplus D_4 \\ P_3 &= 1 \oplus D_1 \oplus D_2 \oplus D_3 \\ P_4 &= 1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus P_1 \oplus P_2 \oplus P_3 \end{aligned}$$

A 4-bit input word $D_4 D_3 D_2 D_1$ is transmitted as the 8-bit code word $D_4 P_4 D_3 P_3 D_2 P_2 D_1 P_1$.

EXAMPLE 14.3—(8,4) Hamming Code

The Hamming (8,4) code word for the input value 0x5 is 0xDC, while the code word for 0x0 is 0x15. ■

Protection of the magazine number and packet number is achieved by forming an 8-bit word in which the magazine number forms the least significant 3 bits and the packet number forms the most significant 5 bits. Each nibble of this 8-bit word is coded using the Hamming (8,4) code, giving a total size for these fields of 16 bits.

ITU-R System B Teletext Page Header The page header (i.e., Packet 0 of each page) also indicates a page subcode, which allows one page to contain a number of subpages, each of which is designed to be displayed individually. There are four subcode components, denoted as S1, S2, S3, and S4, whose sizes are 4, 3, 4, and 2 bits, respectively.

A number of control bits are also transmitted in the page header, whose meanings are shown in Table 14.7.

Table 14.7 Allocation of control bits in Packet 0.

Control bit	Function
C4	Erase page—packets from a previous transmission of the page should be erased from decoder's memory before decoding and storing new transmission.
C5	Indicates a news flash that should be displayed boxed inset into normal video.
C6	Subtitle—should be displayed inset into normal picture when subtitles are selected.
C7	Suppress header—data transmitted in line 0 should be ignored.
C8	Update indicator—data in page has changed since previous transmission.
C9	Interrupted sequence—the associated page is not in numerical order of page sequence.
C10	Inhibit display—data from page should not be displayed.
C11	Magazine serial—set when all pages from one magazine are transmitted before any pages from another magazine.
C12, C13, C14	National character option subset—used to control national character sets.

Table 14.8 Contents of page subcode and control bits.

Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
S1	S2 C4	S3	S4 C5 C6	C7 – C10	C11 – C14

Six bytes are used to transmit the page subcodes and control bits, with each byte protected by the Hamming 8/4 code. The first byte carries S1, the second S2 and C4, the third S3, the fourth S4, C5, and C6, with C7-C10 in byte 5 and C11-C14 in byte 6 as shown in Table 14.8. In each case, leftmost bits are the least significant, and bytes are transmitted least-significant-bit first.

ITU-R System B Teletext Character Set The basic teletext character set (known as the G0 character set), which is based on the ASCII standard, supports minor variations for language-specific characters. The English variant is shown in Table 14.9. The precise appearance of characters is a decoder option, although the teletext standard provides a sample character set based on a 10 × 12 pixel grid.

ITU-R System B Teletext Attributes The presentation of characters may be modified by the selection of other character sets or by changing the attributes of the basic G0 set using *spacing attributes*, a selection of which is shown in Table 14.10.

Attributes labeled “set-after” apply to subsequent character positions; those labeled “set-at” apply from the current character position. Each position occupied

Table 14.9 Basic teletext character set.

D6 D5 D4 D3	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
	D2 D1 D0											
000		(	0	8	@	H	P	X	—	h	p	x
001	!	)	1	9	A	I	Q	Y	a	I	q	y
010	“	*	2	:	B	J	R	Z	b	j	r	z
011	£	+	3	;	C	K	S	←	c	k	s	¼
100	\$	,	4	<	D	L	T	½	d	l	t	
101	%	-	5	=	E	M	U	→	e	m	u	¾
110	&	.	6	>	F	N	V	↑	f	n	v	÷
111	‘	/	7	?	G	O	W	#	g	o	w	■

Table 14.10 Spacing attributes for ITU-R System B teletext.

Code	Function
0x00	Alpha black (“set-after”)—sets the text color to black.
0x01	Alpha red (“set-after”)—sets the text color to red.
0x02	Alpha green (“set-after”)—sets the text color to green.
0x03	Alpha yellow (“set-after”)—sets the text color to yellow.
0x04	Alpha blue (“set-after”)—sets the text color to blue.
0x05	Alpha magenta (“set-after”)—sets the text color to magenta.
0x06	Alpha cyan (“set-after”)—sets the text color to cyan.
0x07	Alpha white (“set-after”)—sets the text color to white—this is the default at the beginning of a packet.
0x08	Flash (“set-after”)—causes foreground pixels subsequent characters to flash between foreground and background colors.
0x09	Steady (“set-at”)—cancels a previous flash attribute for subsequent characters.
0x0A	End box (“set-after”)—terminates a box beginning with the “Start box” attribute.
0x0B	Start box (“set-after”)—Used with subtitles to specify a part of the picture that is to be boxed on top of the normal picture. Characters outside this area are not displayed.
0x0C	Normal size (“set-at”)—cancels a previous double height, double width, or double size code for subsequent characters.
0x0D	Double height (“set-after”)—characters have double height and are stretched into the next row down. Any characters transmitted in these positions are ignored.
0x0E	Double width (“set-after”)—subsequent characters are displayed at double their normal width.
0x0F	Double size (“set-after”)—subsequent characters are displayed at double their normal height and width.
0x18	Conceal (“set-at”)—subsequent characters are to be displayed as spaces until a color code is received. A decoder or user action may reveal these characters.
0x1C	Black background (“set-at”)—sets background color for subsequent characters to black—default at beginning of row.
0x1D	New background (“set-at”)—the current foreground color is adopted as the background color. The foreground color must be changed before subsequently transmitted characters become visible.

by a spacing attribute is usually displayed as a space. The action of an attribute persists until the end of a row or explicit modification by the transmission of another attribute.

ITU-R System B Teletext Syntax An outline of the syntax used by the teletext system is shown in Table 14.11. Each packet begins with the magazine and packet

Table 14.11 Syntax of ITU-R System B Teletext.

Syntax	Number of bits	Mnemonic
teletext_packet() { magazine_packet_number if (packet == 0) { page_number subcode_and_control } if (packet_number <= 24) for (k = n; k < 45; k++) char_or_attribute_byte else { ... } }	16 8 48 8	bslbf bslbf bslbf bslbf

numbers encoded together as two 8/4 Hamming code words. A packet number of zero indicates a new page, for which the page number is transmitted as an 8/4 Hamming code word, followed by the 6-byte subcode and control field (Table 11.8), of which each byte is an 8/4 Hamming code word. For packets with packet number less than or equal to 24, the remaining bytes of each packet are filled with attribute bytes (Table 11.9) or text characters (Table 11.10).

EXAMPLE 14.4—ITU-R System B Teletext

Figure 14.8 shows an example of teletext page, with a black (default color) background and white (default color) text. The first line of text is double height, double width. The second line is double height, whereas the third line is double width, and the last line is normal height and width. The construction of this simple page in page 10 of magazine 4 with all subcode and control bits set to 0 is shown in Table 14.12.

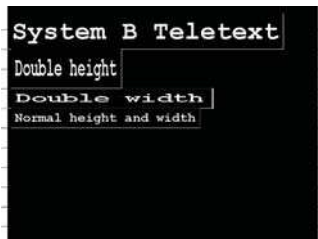


Figure 14.8 Example of teletext page.

Table 14.12 Construction of sample teletext page in Figure 11.8.

Command/data	Transmitted data (expressed as 7-bit hexadecimal words, excluding parity)
Clock run-in and framing	55 55 27
Magazine 4, Packet 0	D0 15
Page number	02 15
Page subcode and control	15 15 15 15 15 15
Set double size attribute	0F
Text "System B Teletext"	53 79 73 74 65 6D 20 42 20 54 65 6C 65 74 65 78 74
Fill remainder of packet	20 20 20 20 20 20 20 20 20 20 20 20 20 20
Clock run-in and framing	55 55 27
Magazine 4, Packet 2	D0 49
Set double height attribute	0D
Text "Double height"	44 6F 75 62 6C 65 20 68 65 69 67 68 74
Fill remainder of packet	20 20
Clock run-in and framing	55 55 27
Magazine 4, Packet 4	D0 25
Set double width attribute	0E
Text "Double width"	44 6F 75 62 6C 65 20 77 69 64 74 78
Fill remainder of packet	20 20
Clock run-in and framing	55 55 27
Magazine 4, Packet 5	D0 33
Text "Normal height and width"	4E 6F 72 6D 61 6C 20 68 65 69 67 68 74 20 61 6E 64 20 77 69 64 74 78
Fill remainder of packet	20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20

In this example, because the first two lines are double height, any data transmitted in Packets 1 or 3 is ignored. The second line of text is therefore transmitted in Packet 2 and the third line in Packet 4 and the fourth line in Packet 5. ■

14.2.2.2. DVB Teletext

DVB allows teletext information to be carried as private data in PES packets of an MPEG-2 stream. The PID of the PES packets used to carry teletext data is defined in the program map table for that service. DVB teletext may be used to provide either the teletext service or as an alternative means of subtitling; this usage is identified within the teletext and also by DVB's teletext descriptor that is carried in the program map table (see Section 11.6.2.3). Multiple streams of teletext may be present in one PID. These can be distinguished by different values of **data_identifier** between 0x10 and 0x1F in the PES packet header. Each teletext stream should use a single value of the **data_identifier**.

The data carried in these packets is decoded by the set-top box, which may provide directly for the display of this data or may simply transcode this data into the VBI of an output analog television signal. In the former case, the set-top box stores the teletext data and provides a user interface to switching between teletext and video and the selection of a particular page of teletext. In the latter case, a teletext decoder in the analog television receiver is responsible for decoding and display of the transcoded teletext data. By restricting the amount of data in each teletext stream to 16 lines per field, DVB guarantees that transcoding of this teletext data in to an analog television signal is always possible.

14.3. ATSC CLOSED CAPTIONING

There are two closed-captioning (i.e., subtitling) systems specified by the ATSC standard, both developed by the Electronic Industry Association. The first is the “Line 21 Data Service” [4], which provides closed captioning for NTSC analog transmissions. The second, specification for advanced television closed captioning (ATVCC), is specifically designed for digital television [5]. Support for closed captioning is mandated by the FCC for all digital television receivers in the United States.

All closed-captioning data in an ATSC transmission is carried as user data in the MPEG-2 video picture header [6], which is intended to ease the task of transcoding between digital and analog signals. Data is carried in quanta of 16 bits, with each picture header able to carry between 0 and 31 quanta, that is, up to 496 bits. It is required that an average data rate of 9600 bit/s be maintained for closed-caption data (if present), which equates to an average of 320 bits per picture with a transmission rate of 30 pictures per second.

The type of closed-captioning data (i.e., Line-21 data service or ATVCC) is indicated within the picture-header user data. ATSC’s caption service descriptor (described in Chapter 12), which may be transmitted in the program map table or event information table, also tells a decoder which subtitling system is used by a particular program.

This section describes both the line-21 data service and ATVCC.

14.3.1. Line 21 Data Service

The Line 21 data service provides a text-based closed-captioning system. Its name comes from the fact that the data for captions in analog television is transmitted in line 21 of each frame (and optionally in line 248), which is part of the VBI. Each field carries two 7-bit characters of data, each of which is protected by 1 parity bit (using odd parity). Seventeen bits are transmitted in each field, beginning with 1 start bit, which is followed by the first 7-bit character (LSB first) and its parity bit, followed by the second 7-bit character (LSB first) and its parity bit. On a standard 4:3 interlaced display, the Line 21 data service can display 15 lines (numbered 1–15),

each containing up to 32 characters. In ATSC digital television, the 2 bytes of the Line 21 data service are carried in the MPEG-2 video bit stream in the **user_data** field of the picture header.

The Line 21 data service supports a number of operating modes, including the following:

- *Text mode.* The text mode displays text in 7–15 rows of the display and displays text immediately after it is decoded. The cursor is initially at the top left position and moves to the right side as text is displayed and down as carriage-return commands are received. When the display is full, it scrolls up one line and places new text on the bottom line. If more than 32 characters are transmitted on one line, the 33rd and subsequent characters overwrite the 32nd character. Text mode is entered on receiving one of the commands “resume text display” or “text restart.”
- *Roll-up captioning.* The bottom 2, 3, or 4 rows of the display contain the caption. New text is always displayed on the bottom line (15), and the display scrolls when this line becomes full or a carriage-return command is received, while the cursor returns to the beginning of line 15. Roll-up captioning mode is entered on receiving one of the commands “roll-up captions, 2-rows,” “roll-up captions, 3-rows,” or “roll-up captions, 4-rows.”
- *Pop-on captioning.* Two text buffers are provided, one for the caption currently being displayed and the other for the caption currently being received. The received caption text is buffered without changing the displayed caption. When an “end of caption” command is received, the buffers swap roles. Four rows of text (either rows 1–4 or 12–15) are supported. This mode is entered when the “resume caption loading” command is received. The “erase nondisplayed memory” is used to remove the contents of the receiver buffer before transmitting its new contents.
- *Paint-on captioning.* One text buffer is provided, into which caption text is both loaded on decoding and simultaneously displayed. This mode is entered on receipt of the “resume direct captioning” command.

All transmitted data is either a control code, a text character, or a null character. All control codes have the form 001*H*XXX XXXXXXXX, which allows them to be distinguished from text characters, all of which have the form 1XXXXXX or 01XXXXXX. Two channels of caption data are supported, distinguished by the 1-bit value of “*H*” in the control code. A null character is used as a filler where no data is to be transmitted.

Control codes consist of 2 bytes of data, which are transmitted twice. There are three types of control codes: preamble address codes, midrow codes, and miscellaneous control codes. Prototypes of the three types of control codes are shown in Table 14.13. *H* is used to distinguish commands associated with the two

Table 14.13 Prototypes of control codes.

Code	Prototype
Preamble address code	$0\ 0\ 1\ H\ L_3\ L_2\ L_1\ 1\ L_0\ N\ C_2\ C_1\ C_0\ U$
Midrow code	$0\ 0\ 1\ H\ 0\ 0\ 1\ 0\ 1\ 0\ C_2\ C_1\ C_0\ U$
Miscellaneous control code	$0\ 0\ 1\ H\ 1\ 0\ F\ 0\ 1\ 0\ M_3\ M_2\ M_1\ M_0$

captioning channels supported. U is set to one if the caption is to be underlined. If N is zero, $C_2\ C_1\ C_0$ determines the color in which text is displayed in accordance with Table 14.14. If N is one, text is displayed in white and indented by $4n$ spaces, where n is the 3-bit, unsigned binary number represented by $C_2\ C_1\ C_0$ spaces. $L_3\ L_2\ L_1\ L_0$ determines the row number to which text is addressed for the preamble control code, following the assignment set out in Table 14.15. The commands associated with the miscellaneous control codes are represented by the value of $M_3\ M_2\ M_1\ M_0$ and are defined in Table 14.16. F is zero if the command is transmitted in line 21, one if the command is transmitted in line 248.

Miscellaneous control codes are used to set the operating mode of the closed-captioning decoder and sometimes to control its operation within this mode (e.g., to flip memories in the pop-on captioning mode).

Preamble address codes are always used at the beginning of a line of text, except in the caption-roll-up and text modes. In these latter cases, the line on which text is displayed is determined automatically by the mode. In other modes, the specification of the line number in the preamble address code is used by the decoder to position text in the display. Color attributes are automatically reset at the beginning of each line.

Midrow codes are used within a line of text to alter its attributes and may also be used at the beginning of a line of text in caption-roll-up and text modes. Changing the color automatically turns off italics; changing the italics has no effect on the color.

Table 14.14 Color codes.

$C_2\ C_1\ C_0$	Color
000	White
001	Green
010	Blue
011	Cyan
100	Red
101	Yellow
110	Magenta
111	<i>Italic</i>

Table 14.15 Codes for row addresses.

$L_3 L_2 L_1 L_0$	Row number
0000	11
0001	Not used.
0010	1
0011	2
0100	3
0101	4
0110	12
0111	13
1000	14
1001	15
1010	5
1011	6
1100	7
1101	8
1110	9
1111	10

Table 14.16 Commands for miscellaneous control codes.

$M_3 M_2 M_1 M_0$	Command
0000	Resume caption loading
0001	Backspace
0010	Reserved
0011	Reserved
0100	Delete to end of row
0101	Roll-up captions, 2 rows
0110	Roll-up captions, 3 rows
0111	Roll-up captions, 4 rows
1000	Flash on
1001	Resume direct captioning
1010	Text restart
1011	Resume text display
1100	Erase displayed message
1101	Carriage return
1110	Erase nondisplayed memory
1111	End of caption (flip memories)

Table 14.17 Basic character set of EIA-608 closed captioning.

D6 D5 D4 D3												
	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
D2 D1 D0												
000		(	0	8	@	H	P	X	ú	h	p	x
001	!	)	1	9	A	I	Q	Y	a	i	q	y
010	“	á	2	:	B	J	R	Z	b	j	r	z
011	#	+	3	;	C	K	S	[	c	k	s	ç
100	\$	,	4	<	D	L	T	é	d	l	t	÷
101	%	-	5	=	E	M	U	]	e	m	u	Ñ
110	&	.	6	>	F	N	V	í	f	n	v	ñ
111	‘	/	7	?	G	O	W	ó	g	o	w	■

Characters for display may follow a preamble address code or a midrow control code. The basic character set for captions is shown in Table 14.17. These codes mostly follow the ASCII standard.

EXAMPLE 14.5—Line-21 Data Service

What data is transmitted by the Line-21 data service to display the text:

Hello world

Line-21 data standard is published

by the EIA

with the first and last lines colored green and the middle line colored blue in the “roll-up captions, 2 rows” mode.

Table 14.18 shows the sequence of transmitted data required in order to transmit the text. The transmission begins with the miscellaneous command to set the transmission mode. This has the binary form $001H10F010M_3M_2M_1M_0$. For this command, $M_3M_2M_1M_0 = 0101$, so if $H = 0$ and $F = 0$, the binary form of the command is $0010100\ 00100101$, which is equivalent to 14 25 in hexadecimal.

Once the transmission mode is set, a midrow command is used to set the attributes of the row of text to green. After the line of text is transmitted, a carriage-return command moves the display to the next line of text, and two midrow commands are required to set the text to blue italics. Following some text, a midrow command setting the color is used to turn off italics, after which more text is transmitted on the same line. This line is followed by another carriage-return command, a midrow command to begin the next line and the text of the line.

Table 14.18 Sequence of transmitted data for Line-21 example.

Command/data	Transmitted data (expressed as 7-bit hexadecimal words, excluding parity)
Roll-up captions, 2-rows	14 25
Midrow command, green	11 22
“Hello world”	48 65 6C 6C 6F 20 77 6F 72 6C 64
Carriage return	14 2D
Midrow command, blue	11 24
Midrow command, italics	11 2E
“Line-21 Data Standard ”	4C 69 6D 65 2D 32 31 20 44 61 74 61 20 43 74 61 6D 64 61 72 64 20
Midrow command, blue	11 24
“is published by”	69 72 20 70 75 62 6C 69 73 68 65 64 20 62 79
Carriage return	14 2D
Midrow command, green	11 22
“the EIA”	74 68 65 20 45 49 41 2E
Carriage return	14 2D

The complete stream of characters to be transmitted is therefore 14 25 11 22 48 65 6C 6C 6F 20 77 6F 72 6C 64 14 2D 11 24 11 2E 4C 69 6D 65 2D 32 31 20 44 61 74 61 20 43 74 61 6D 64 61 72 64 20 11 24 69 72 20 70 75 62 6C 69 73 68 65 64 20 62 79 14 2D 11 22 74 68 65 20 45 49 41 2E 14 2D. ■

14.3.2. Advanced Television Closed Captioning

The closed captioning used for all ATSC services is defined by EIA-708-B [7] and is known as ATVCC. This caption system supports text captions that are displayed in windows on the screen. The structure of the caption services associated with a video stream is defined by the ATVCC service directory carried in the caption service descriptor in the program map table and/or event information table.

All ATSC television services carry ATVCC data. Each second of video carries a nominal 9600 bits, which is normally allocated as 20 bytes per frame with a frame rate of 60 Hz. This constant-bit-rate approach means that insertion of closed-captioning data into a precoded video stream can be accomplished without the need for retiming of the bitstream (with its accompanying difficulties of recoding the clock references in the transport stream).

All ATVCC data is transmitted in packets, each of which may overlap frame boundaries. Data is inserted into the video bit stream in “frame order.”

The ATVCC specification assumes that the underlying ATSC system provides a nominally error-free channel. Consequently, no channel coding is provided by ATVCC.

ATVCC allows several different captioning services to be multiplexed together, allowing the viewer to select the most appropriate. This would allow, for example, subtitles for a movie to be carried in a number of different languages, while at the same time providing a separate captioning service for hearing-impaired viewers.

Data from each captioning service is displayed in windows, which can be positioned on the video display. The ATVCC bit stream includes information that describes the attributes of these windows (such as size and color) and directs text into a particular window for display.

14.3.2.1. Caption Display Model

In the ATVCC channel, data is assigned to one of 63 caption services. Caption service 1 is the primary caption service, whereas caption service 2 is known as the secondary language service. It is intended that a viewer selects one particular caption service to be decoded. Support for multiple caption services means that simultaneous support can be provided for several different secondary languages, as well as providing captions for the hearing impaired.

All caption data are displayed as text in windows, very much like the windows provided by a personal computer. Each window is displayed over the top of the program's video material. A caption service may define up to eight windows.

Each caption window is assigned a number of attributes, including its size and background color. Each window has an associated pen, which also has attributes that control the properties of characters as they are displayed.

This section presents the ATVCC models for the screen, window and pen, and the method used for specifying color.

Color Representation Color in ATVCC is specified using a palette of red, green, and blue. Each color is specified with a resolution of 2 bits, making possible a total of 64 distinct colors. For example, white is specified by the (r, g, b) values (3,3,3), black by (0,0,0), and bright green by (0,3,0).

In some circumstances, opacity can be specified. Opacity is defined by a 2-bit number, with the meanings shown in Table 14.19. "Solid" means that pixels lying behind the object are not visible; "translucent" means that pixels lying behind the object are partially visible, and transparent means that pixels lying behind the object are fully visible (and the object itself is not visible). "Flashing" means that the object alternates between solid and transparent.

Caption Screen Coordinate System ATVCC defines a rectangular grid of cells that sits over the top of the displayed picture. Normally, this grid does not cover the whole of the displayed picture, leaving margins at the left, right, top, and bottom

Table 14.19 Opacity values associated with ATVCC colors.

Opacity	Meaning
0	Solid
1	Flashing
2	Translucent
3	Transparent

of the screen. For a 16:9 aspect ratio display, the grid is 210 horizontal cells by 75 vertical cells. For a 4:3 aspect ratio, the grid is 160 cells horizontally and 75 cells vertically.

Grid coordinates are specified as pairs of integers, with the top left cell being (0, 0). The first number specifies the horizontal location (column) of the cell; the second specifies the vertical location (row).

Caption Windows Each service may define up to eight caption windows. The position of a window on the screen may be specified using either relative or absolute positioning.

All caption data are displayed in windows, which occupy a part of the display screen. A window must be defined before caption data can be displayed in it. All windows are rectangular in shape. For each window, the following parameters are defined:

- *Priority*. The relative priority of two windows determines which one appears on top if they overlap. Eight different priorities are supported. A lower value of the priority indicates a higher priority.
- *Anchor identifier*. The anchor of a window is the point in the window that is used to specify its location. The anchor has two components: one horizontal and the other vertical. The horizontal component may refer to the left, middle, or right side of the window. The vertical component may refer to the top, middle, or bottom portion of the window. In total, there are therefore nine different possible types of anchor.
- *Anchor location*. The anchor location specifies a horizontal and vertical value for the location of the window's anchor. Both horizontal and vertical values are specified in terms of the screen's grid of cells.
- *Window size*. The size of the window is specified as the number of rows of characters contained in the window vertically and the number of columns horizontally. The actual displayed size of the window therefore depends on the font size.
- *Row lock and column lock*. These parameters are used to specify whether or not the display is allowed to change the number of rows and columns in a window on the fly.
- *Visibility*. The visibility of a window may be set to 1 (displayed) or 0 (hidden).
- *Row and column count*. The row and column count specify the width and height of the window, respectively.
- *Window style id*. Allows window attributes to be selected from one of the seven predefined styles when the window is created.
- *Pen style id*. The attributes associated with the pen in the current window may be specified by the pen style identifier.
- *Justification*. Displayed text may be left, right, center, or full justified.
- *Print direction*. The direction in which characters are written may be left-to-right, right-to-left, top-to-bottom, or bottom-to-top.

- *Scroll direction.* When carriage returns occur, the scroll direction controls the direction in which scrolling occurs. The options for scroll direction are the same as for print direction. Horizontal scrolling is permitted only for vertical print directions. Likewise, vertical scrolling is only allowed for horizontal print directions.
- *Word wrap.* Word wrap specifies whether the decoder is allowed to force words onto a new line where a transmitted line is too long to be displayed in the window.
- *Display effect.* The display effect controls the use of snap, wipe, and fade when a window is displayed or hidden.
- *Effect direction.* Where an effect such as wipe is used in displaying or hiding a window, the direction of this effect can be specified.
- *Effect speed.* The effect speed determines the time for a window to transition fully between the hidden and displayed states.
- *Fill color.* The fill color is the color of the window background, using the color format defined in section “Color Representation.”
- *Fill opacity.* The fill opacity is the opacity of the window fill as specified in Table 14.19.
- *Border type.* Window borders may be used to surround windows. These borders may be raised, depressed, uniform, or drop shadowed.
- *Border color.* The border color is the color of the window border, using the color format defined in section “Color Representation.”

Caption Pen The caption pen is a notional pen that draws new text to be displayed onto a window. The pen has a number of attributes, which control the properties of characters that are moved to the display. Changing the pen attributes only affects future characters displayed. Characters already in the display remain unchanged.

The attributes associated with the caption pen are described in section “ATVCC Pen Commands.”

14.3.2.2. Caption Packet Format

Each caption packet consists of a 1-byte header and between 1 and 127 bytes of caption data. The packet header contains a 2-bit sequence number, used by the receiver to detect lost packets, and a 6-bit packet size code. The total length of the packet (including header) is twice the value specified in the packet size code.

14.3.2.3. Service Block Payload

Within each packet, caption data is carried in service blocks. Each service block carries data for one service and cannot overlap a caption packet boundary. A packet carries one or more service blocks. Each service block consists of a 1 or 2 byte header and a payload, which carries between 0 and 31 bytes of data for one service.

Table 14.20 Assignment of codes in ATVCC service block payload.

Most significant bits of byte	Use
0	Subset of ASCII control codes.
1	
2	Displayed characters, using a slightly modified form of the standard ASCII codes.
3	
4	
5	
6	
7	Caption control codes (see Section 14.3.2.4).
8	
9	
A	Additional characters following the ISO 8859-1 Latin character set and room for future expansion.
B	
C	
D	
E	
F	

For services 1–7, the service block has a 1-byte header, consisting of a 3-bit service number and a 5-bit payload length. For services 8–63, the first header byte specifies a service number of zero, and a second header byte carries a 6-bit service number field. The remaining 2 bits of this second byte are set to 0.

Each payload byte of a service block carries one text character or forms part of a control code. The decoder uses the most significant 4 bits of a byte to determine whether it is a character to be displayed or a control code as set out in Table 14.20.

14.3.2.4. ATVCC Commands

ATVCC commands are used to control the operation of the closed-captioning system in the receiver. They are grouped into three classes:

- *Windows commands.* Windows commands are used to create, delete, clear, display, and hide closed-captioning windows. They can also be used to set the attributes of windows, including their color and position.
- *Pen commands.* ATVCC pen commands are used to control the attributes, color, and location of the pen.
- *Synchronization commands.* The ATVCC synchronization commands are used to program delays between when caption data is received and displayed.

All of the commands in each of these three classes are discussed in the remainder of this section.

Table 14.21 Structure of the ATVCC SetCurrentWindow command.

	Bit number							
Byte	7	6	5	4	3	2	1	0
0	1	0	0	0	0	<i>id</i>		

ATVCC Windows Commands The ATVCC windows commands control the operation of windows, allowing the current window to be set, new windows to be defined, the attributes of windows to be changed, the display status (displayed or hidden) of windows to be altered, and windows to be deleted.

SetCurrentWindow The SetCurrentWindow command is a 1-byte command that selects one of the eight windows to be the current window. This window must have already been defined using the DefineWindow command.

The structure of the ATVCC SetCurrentWindow command, including its argument, is shown in Table 14.21. The number of the window (between 0 and 7) chosen to be the new current window is specified as the 3-bit *id* value.

DefineWindow The ATVCC DefineWindow command is a 7-byte command that is used to create a new window. The structure of the ATVCC DefineWindow command, including its arguments, is shown in Table 14.22. The type (*ap*) and horizontal and vertical coordinates (*ah* and *av*, respectively) of the anchor point for the window are specified, along with its priority (*p*), relative positioning (*rp*), row count (*rc*), column count (*cc*), row lock (*rl*), column lock (*cl*), visibility (*v*), window style id (*ws*), and pen style id (*ps*). The available values for each of these parameters are set out in previous sections.

ClearWindows The structure of the ATVCC ClearWindows command is a 2-byte command that specifies a list of windows whose contents are to be cleared. This command can be used to clear simultaneously between one and eight windows.

Table 14.22 Structure of the ATVCC DefineWindow command.

	Bit number							
Byte	7	6	5	4	3	2	1	0
0	1	0	0	1	1		<i>id</i>	
1	0	0	<i>v</i>	<i>rl</i>	<i>cl</i>		<i>p</i>	
2	<i>rp</i>				<i>av</i>			
3				<i>ah</i>				
4		<i>ap</i>				<i>rc</i>		
5	0	0			<i>cc</i>			
6	0	0		<i>ws</i>			<i>ps</i>	

Table 14.28 Structure of the ATVCC Set Window Attributes command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	1	0	1	1	1
1		<i>fo</i>		<i>fr</i>		<i>fg</i>		<i>fb</i>
2	<i>bt</i>			<i>br</i>		<i>bg</i>		<i>bb</i>
3		<i>ww</i>		<i>pd</i>		<i>sd</i>		<i>j</i>
4			<i>es</i>			<i>ed</i>		<i>de</i>

ATVCC Pen Commands. The three ATVCC pen commands are used to set the attributes, color, and location of the pen.

SetPenAttributes The ATVCC SetPenAttributes command is used to set the attributes associated with the pen for the current window. These attributes include the pen size, font style, text tag, and control of italics and underlining. Pen attributes for a window can be changed at any time and remain in effect until either the next SetPenAttributes command or the window is closed.

The structure of the ATVCC SetPenAttributes command, including its arguments, is shown in Table 14.29. The arguments to the SetPenAttributes command are as follows:

- *Text tag (tt)*. Sixteen caption function tags are defined. They are used to indicate that the text displayed is dialog, source or speaker identification, electronically synthesized voice, dialog in a language different to the primary language, voiceover, and a range of other effects.
- *Offset (o)*. Displayed text may be one of superscript, subscript, or normal.
- *Pen size (s)*. Specifies whether the pen is small, medium, or large.
- *Italics (i)*. A value of 1 indicates the use of italics.
- *Underline (u)*. A value of 1 indicates that text is to be underlined.
- *Edge type (et)*. Edge types for the text include raised, depressed, and uniform.
- *Font style (fs)*. Although a specific font cannot be specified, font styles such as the use of serifs, mono versus proportional spacing, and the use of small capitals can be specified.

Table 14.29 Structure of the ATVCC SetPenAttributes command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	1	0	0	0	0
1			<i>tt</i>		<i>o</i>		<i>s</i>	
2	<i>i</i>	<i>u</i>		<i>et</i>		<i>fs</i>		

Table 14.30 Structure of the ATVCC SetPenColor command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	1	0	0	0	1
1	<i>fo</i>		<i>fr</i>		<i>fg</i>		<i>fb</i>	
2	<i>bo</i>		<i>br</i>		<i>bg</i>		<i>bb</i>	
3	0	0	<i>er</i>		<i>eg</i>		<i>eb</i>	

SetPenColor The ATVCC SetPenColor command is used to set the color that is used for characters that are written to the window. Three colors are specified: the foreground color, the background color, and the edge color, each specified using the format described in section “Color Representation.”

The structure of the ATVCC SetPenColor command, including its arguments is shown in Table 14.30. The red, green, and blue values for the text foreground color are defined by *fr*, *fg*, and *fb*, respectively, with the foreground opacity defined by *fo*. Similarly, the red, green, and blue values for the text background color are defined by *br*, *bg*, and *bb* and the background opacity by *bo*. The color for the edge of the text is specified by *er*, *eg*, and *eb*. The opacity of the text edge is the same as the foreground.

SetPenLocation The ATVCC SetPenLocation command sets the location at which the next text is written to the current caption window (known as the *pen location*). This command takes two arguments, which are the row address (*r*) and the column address (*c*) to which the pen is to be moved. The structure of the ATVCC SetPenLocation command, including its arguments, is shown in Table 14.31.

A conflict exists between the window justification and the location specified by the SetPenLocation command if the justification is not left. The current window justification is not left:

- if the print direction is left-right or right-left, the column value *c* is ignored, or
- if the print direction is top-bottom or bottom-top, the row value *r* is ignored.

Table 14.31 Structure of the ATVCC SetPenLocation command.

	Bit number							
Byte	7	6	5	4	3	2	1	0
0	1	0	0	1	0	0	1	0
1	0	0	0	0	r			
2	0	0	c					

Table 14.32 Structure of the ATVCC Delay command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	0	1
1	<i>t</i>							

ATVCC Synchronization Commands The ATVCC synchronization commands are used to control the decoding of caption services. They can be used to either define a delay between receipt of data and its decoding or to reset the decoder.

Delay The ATVCC Delay command is a 2-byte command, which suspends interpretation of the current caption service’s command input buffer for *t* tenths of a second. Interpretation recommences when one of the following occurs:

- the specified time delay expires,
- a DelayCancel command is received,
- the service’s input buffer becomes full, or
- a Reset command is received for the service.

The structure of the ATVCC Delay command, including its argument, is shown in Table 14.32.

DelayCancel The ATVCC DelayCancel command is a 1-byte command, which cancels all active delays in the caption decoder. It takes no arguments. The structure of the ATVCC DelayCancel command is shown in Table 14.33.

Reset. The ATVCC Reset command is a 1-byte command that reinitializes the caption service for which it is received. It takes no arguments. The structure of the ATVCC Reset command is shown in Table 14.34. The reset command causes all of the service’s windows to be deleted and removed from the display, all attributes of these windows and their pens to be removed and the service input buffer to be flushed.

Table 14.33 Structure of the ATVCC DelayCancel command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	1	0

Table 14.34 Structure of the ATVCC Reset command.

Byte	Bit number							
	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	1	1

- 14.10 Encode the following set of information into ITU-R System B Teletext packets, expressing the encoded data as hexadecimal numbers: Magazine 3, packet 0, page 4, page subcode and control 0x15 0x15 0x15 0x15 0x15 0x15, double size, blue, “This is a test.”
- 14.11 Encode the following set of information into ITU-R System B Teletext packets, expressing the encoded data as hexadecimal numbers: Magazine 2, packet 1, double height, green, “of manual,” blue, “encoding.”
- 14.12 Encode the following set of information into ITU-R System B Teletext packets, expressing the encoded data as hexadecimal numbers: Magazine 1, packet 20, double width, white, “which is,” blue, flash on, “quite painful.”
- 14.13 Encode the following data using the Line-21 format (with parity bit set to 0), expressing the encoded data as hexadecimal numbers: Roll-up captions (2 rows) mode, blue, “This is a simple closed,” carriage return, blue, “that can,” italic, “be hand-encoded.”
- 14.14 Encode the following data using the Line-21 format (with parity bit set to 0), expressing the encoded data as hexadecimal numbers: Text mode, row 5, yellow, “One,” row 10, blue, italic, “two.”
- 14.15 Decode the following of Line-21 hexadecimal data stream: 14 2B 53 61 6D 70 6C 65 11 26 74 65 78 74.
- 14.16 Decode the following of Line-21 hexadecimal data stream: 14 27 11 2E 4D 6F 72 65 14 2D 74 65 78 74.

REFERENCES

1. *Digital Video Broadcasting (DVB); DVB subtitling system, ETS 300 743*, Sophia Antipolis: ETSI, 1997.
2. The complete specification for System B teletext can be found in:
ITU-R Recommendation 653, *System B, 625/50 Television Systems*; or
Enhanced Teletext Specification, ETS 300 706, Sophia Antipolis: ETSI, 1997.
3. *Digital Video Broadcasting (DVB); Specification for conveying ITU-R System B Teletext in DVB Bitstreams, EN 300 472*, Sophia Antipolis: ETSI, 1997.
4. EIA-608, *Recommended Practice for Line 21 Data Service*, Electronic Industry Association.
5. EIA-708B, *Specification for Advanced Television Closed Captioning (ATVCC)*, Electronic Industry Association, December 1999.
6. See: ATSC Standard A/53A, ATSC Digital Television Standard, Advanced Television Systems Committee, April 2001, Annex D, pp. 46–57; ATSC Document A/54, *Guide to the Use of the ATSC Digital Television Standard*, Advanced Television Systems Committee, October 1995, pp. 96–135.
7. EIA-708B, *Specification for Advanced Television Closed Captioning (ATVCC)*, Electronic Industry Association, December 1999.